Prediction of Babar Azam's ODI Batting Performance Using Decision Tree and Prolog

Student Name: Musharraf Hussain
Submission Date: Jun 1, 2026 12:00 AM
Course: Artificial Intelligence

1. Objective

The main objective of this project is to determine whether Babar Azam's batting performance in One Day International (ODI) matches can be predicted using machine learning techniques. Babar Azam is one of Pakistan's most successful batsmen and has played numerous ODI matches throughout his career. His consistent performances make him an ideal player for analyzing and predicting batting outcomes.

In this project, we use the Decision Tree classification algorithm to predict Babar Azam's batting performance and classify it into one of the following categories:

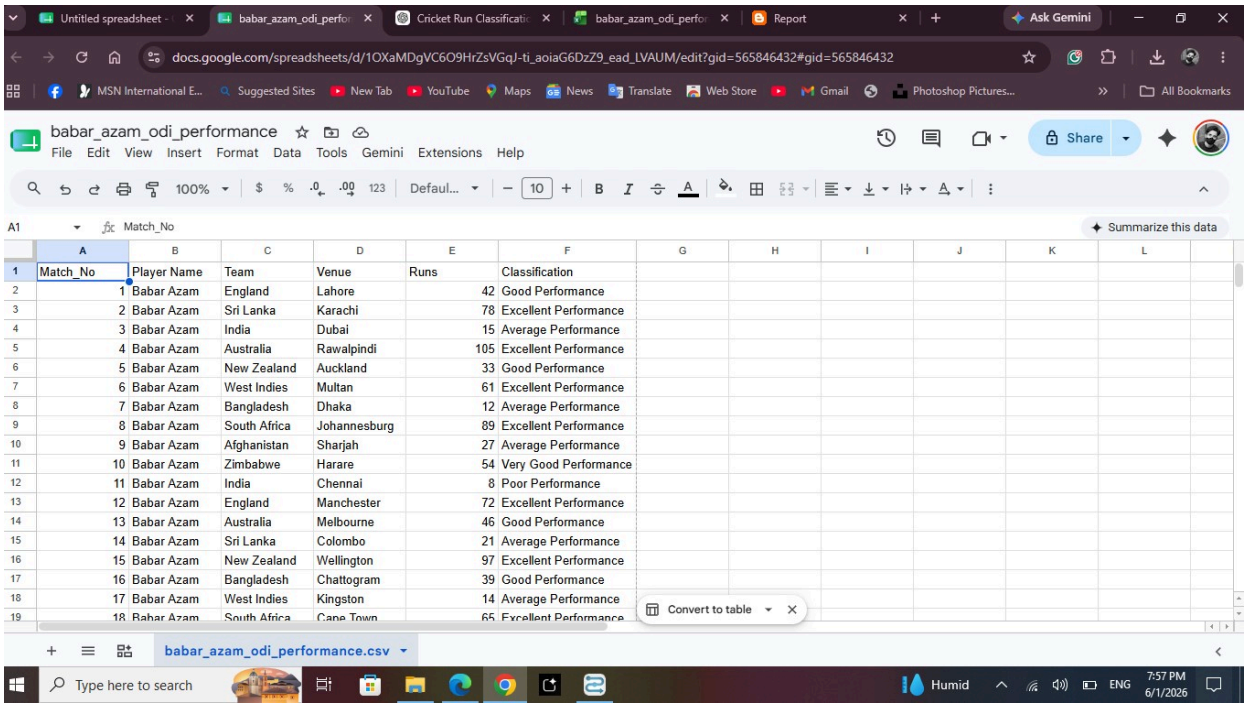
- Excellent Performance
- Very Good Performance
- Good Performance
- Average Performance
- Poor Performance

The dataset consists of 50 ODI match records. Each record contains information related to match conditions and batting results. After training the model, decision rules are extracted from the Decision Tree and implemented in Prolog so that the system can perform automated reasoning and predict performance based on the given inputs.

The columns used in this project are:

1. Player Name
2. Opponent Team
3. Venue
4. Runs Scored
5. Performance Classification

The goal is to analyze how factors such as the opponent team and venue influence Babar Azam's batting performance and to build an intelligent system capable of predicting his performance category in future ODI matches.



Match_No	Player Name	Team	Venue	Runs	Classification
1	Babar Azam	England	Lahore	42	Good Performance
2	Babar Azam	Sri Lanka	Karachi	78	Excellent Performance
3	Babar Azam	India	Dubai	15	Average Performance
4	Babar Azam	Australia	Rawalpindi	105	Excellent Performance
5	Babar Azam	New Zealand	Auckland	33	Good Performance
6	Babar Azam	West Indies	Multan	61	Excellent Performance
7	Babar Azam	Bangladesh	Dhaka	12	Average Performance
8	Babar Azam	South Africa	Johannesburg	89	Excellent Performance
9	Babar Azam	Afghanistan	Sharjah	27	Average Performance
10	Babar Azam	Zimbabwe	Harare	54	Very Good Performance
11	Babar Azam	India	Chennai	8	Poor Performance
12	Babar Azam	England	Manchester	72	Excellent Performance
13	Babar Azam	Australia	Melbourne	46	Good Performance
14	Babar Azam	Sri Lanka	Colombo	21	Average Performance
15	Babar Azam	New Zealand	Wellington	97	Excellent Performance
16	Babar Azam	Bangladesh	Chattogram	39	Good Performance
17	Babar Azam	West Indies	Kingston	14	Average Performance
18	Babar Azam	South Africa	Cape Town	65	Excellent Performance

2. Data Collection Source and Procedure

The data used in this project was collected from the ESPNcricinfo website, which provides detailed statistics and records of international cricket players. The dataset focuses on Babar Azam's ODI batting performances. Information such as runs

scored, opponent team, venue, and match details was gathered from the available records.

For this project, a dataset consisting of 50 ODI match performances was created. Each row in the dataset represents one ODI match played by Babar Azam. The collected data was then analyzed to determine his batting performance under different match conditions, including various opponents and venues.

Based on the number of runs scored in each match, the batting performances were classified into five categories:

- Excellent Performance
- Very Good Performance
- Good Performance
- Average Performance
- Poor Performance

The dataset contains the following attributes:

1. Row Number
2. Player Name
3. Opponent Team
4. Venue
5. Runs Scored
6. Performance Classification

After organizing and classifying the data, the dataset was saved in CSV (.csv) format. The CSV file was then imported into Altair AI Studio for data preprocessing, model training, and evaluation. The Decision Tree algorithm was used to classify and predict Babar Azam's batting performance based on the selected attributes.

3. Selection and Justification of ML Technique

The machine learning technique used in this project is **Supervised Learning**. The reason for selecting this technique is that every record in the dataset already contains the correct output label. For each ODI match played by Babar Azam, the number of runs scored is known, and the performance category has been assigned based on predefined criteria. Therefore, the model can learn from historical data and use that knowledge to predict the performance category of future matches.

This project falls under the category of a **Classification Problem** rather than a Regression Problem. The objective is not to predict the exact number of runs scored by Babar Azam, but rather to classify his batting performance into one of several predefined categories. Based on the runs scored, each match is classified as:

- Excellent Performance
- Very Good Performance
- Good Performance
- Average Performance
- Poor Performance

Since the target attribute (Performance) consists of categorical values instead of continuous numerical values, classification is the most suitable machine learning approach for this problem.

To perform the classification task, the **Decision Tree Algorithm** is used. Decision Trees are easy to understand, generate human-readable rules, and provide clear explanations for predictions. Furthermore, the rules extracted from the Decision Tree can be directly implemented in Prolog, allowing the system to perform logical reasoning and automatically predict Babar Azam's batting performance based on the input attributes.

4. Selection of Specific Algorithm

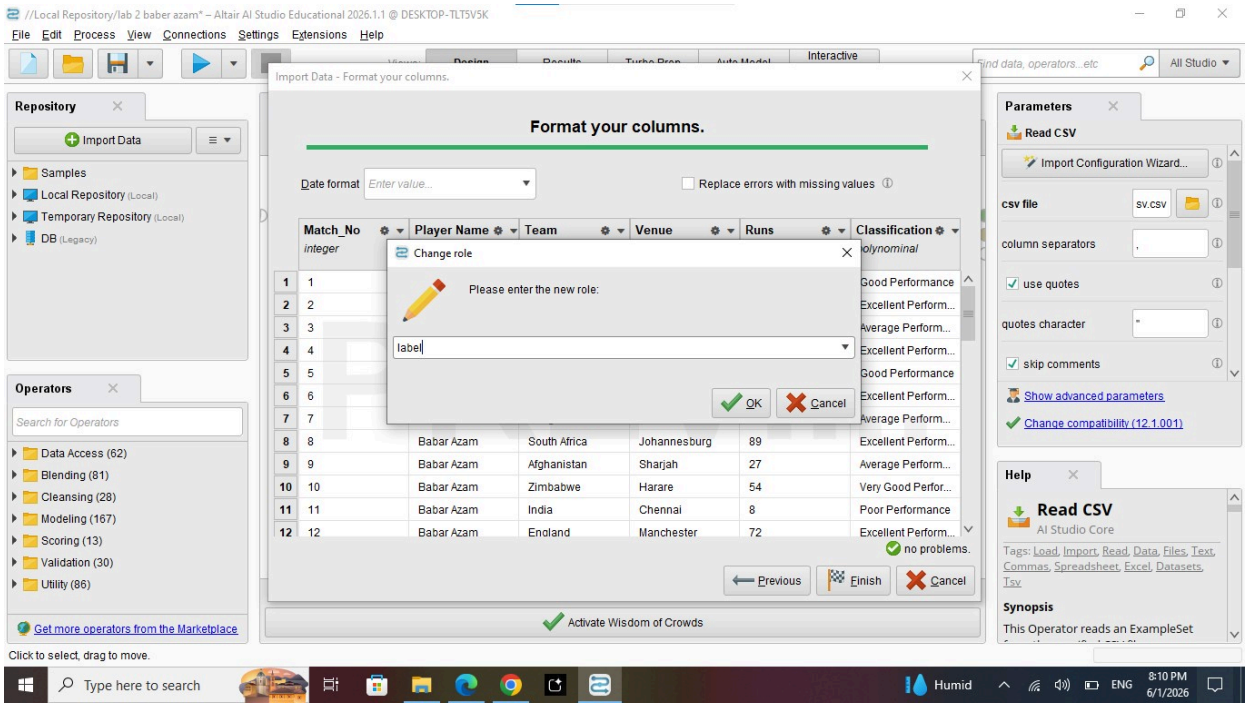
The specific algorithm selected for this project is the **Decision Tree Algorithm**. This algorithm was chosen because the dataset contains categorical information

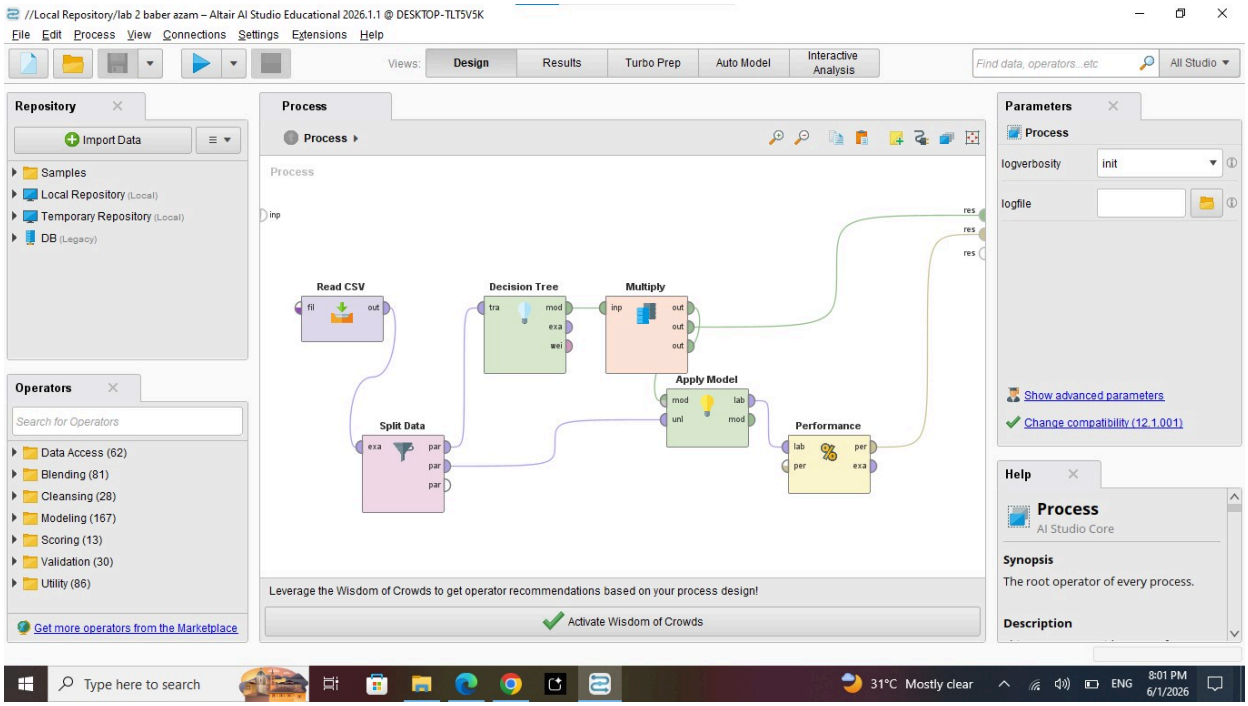
and performance classes that can be easily represented in a tree structure. The objective of the project is to classify Babar Azam's ODI batting performance into categories such as **Excellent Performance, Very Good Performance, Good Performance, Average Performance,** and **Poor Performance**.

Decision Trees are highly effective for classification tasks because they automatically identify the most important attributes that help distinguish between different classes. During training, the algorithm evaluates all available attributes and selects the one with the highest Information Gain as the root node. It then continues splitting the data into smaller subsets until each branch leads to a specific performance category.

Another reason for selecting the Decision Tree algorithm is its simplicity and interpretability. The generated tree can be easily understood and visualized, allowing us to analyze how different factors influence Babar Azam's batting performance. Furthermore, the decision rules extracted from the tree can be directly converted into IF-THEN rules and implemented in Prolog. This makes it easier to develop a reasoning system capable of predicting performance based on the given match conditions.

Therefore, the Decision Tree algorithm is the most suitable choice for this project because it provides high classification accuracy, produces understandable rules, and supports the integration of machine learning with symbolic AI techniques such as Prolog.



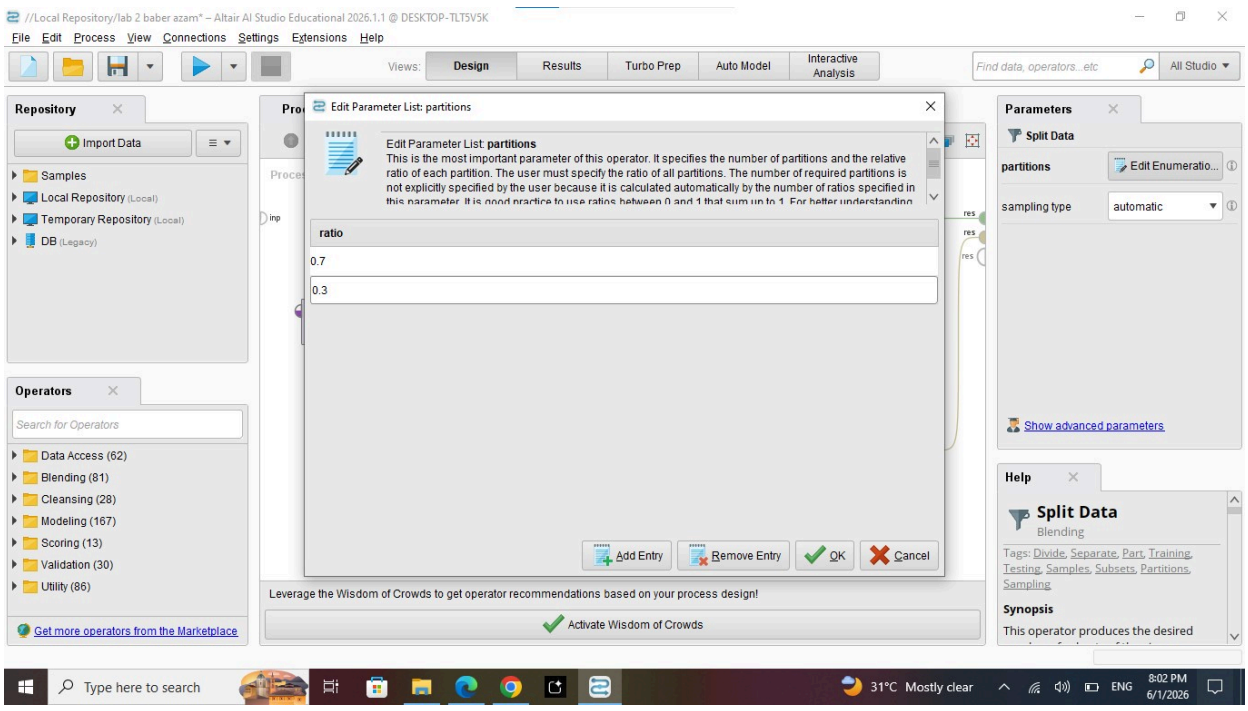


5. Division of Dataset for Learning and Evaluation

In my cricket run classification dataset, the data is divided into two main parts for model development and evaluation. The first part is the training dataset, which contains 70% of the total data, approximately 50 records. This portion is used to train the Decision Tree model so that it can learn patterns and relationships between different features such as match conditions, player performance, and run categories (Low, Medium, High).

The second part is the testing dataset, which contains 30% of the total data, approximately 70 records. These records are kept separate and are not shown to the model during training. The testing data is used only to evaluate the performance and accuracy of the model by checking how well it predicts unseen data.

Furthermore, the dataset is carefully balanced among different performance categories such as Low, Medium, and High runs to ensure fair learning and to avoid bias toward any single class. This balanced distribution helps the model generalize better and improves the reliability of the classification results.



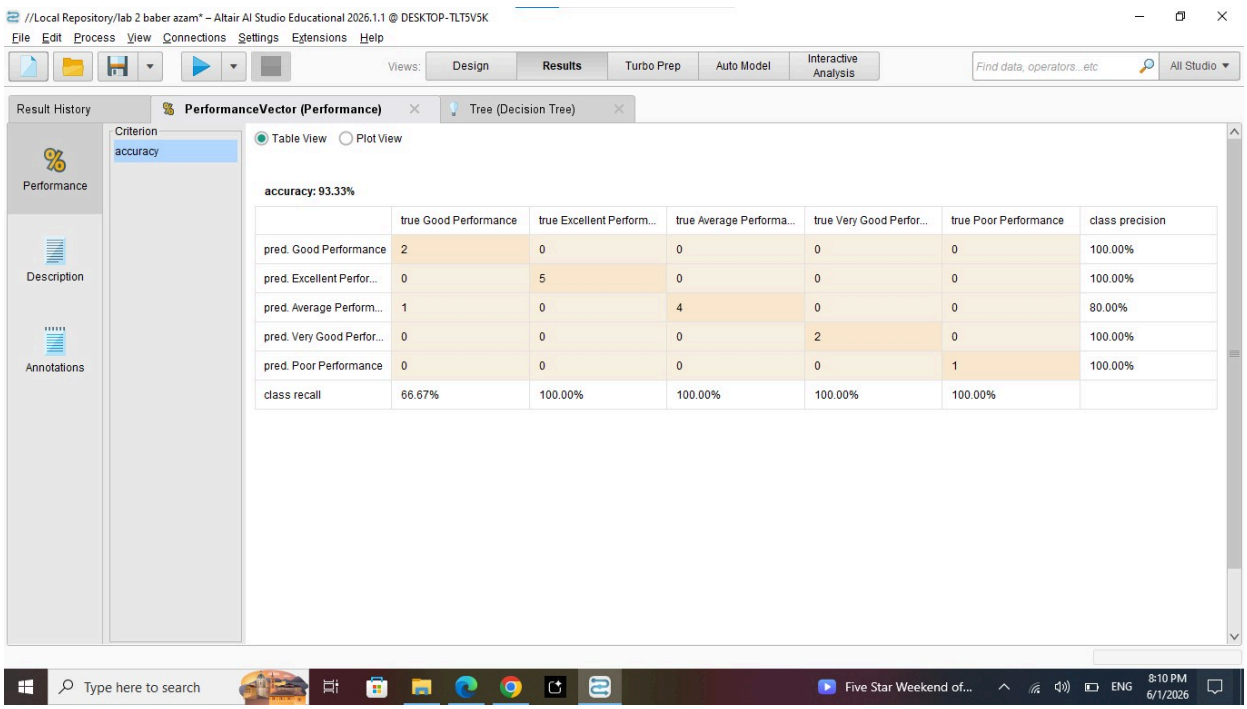
6. Model Visualisation and Description

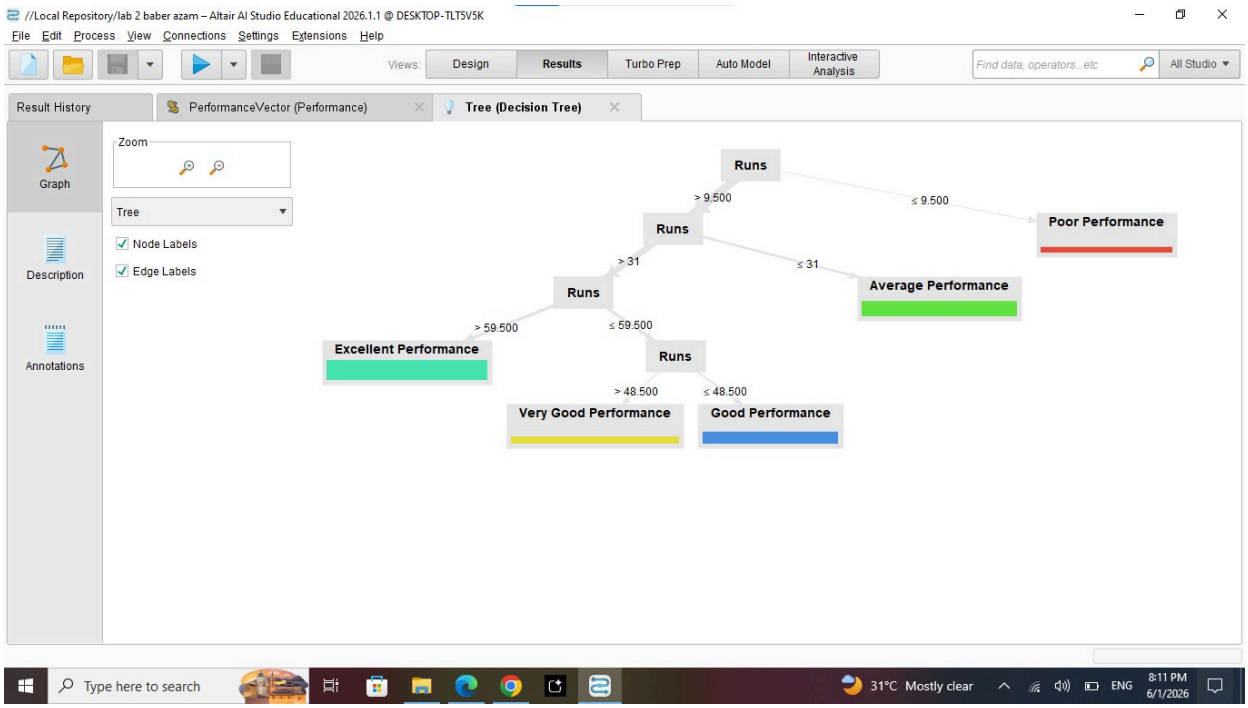
In Altair AI Studio, I trained a Decision Tree model for cricket run classification. The model automatically selected **Economy Rate** as the first and most important decision point because it gives the most useful information for predicting performance.

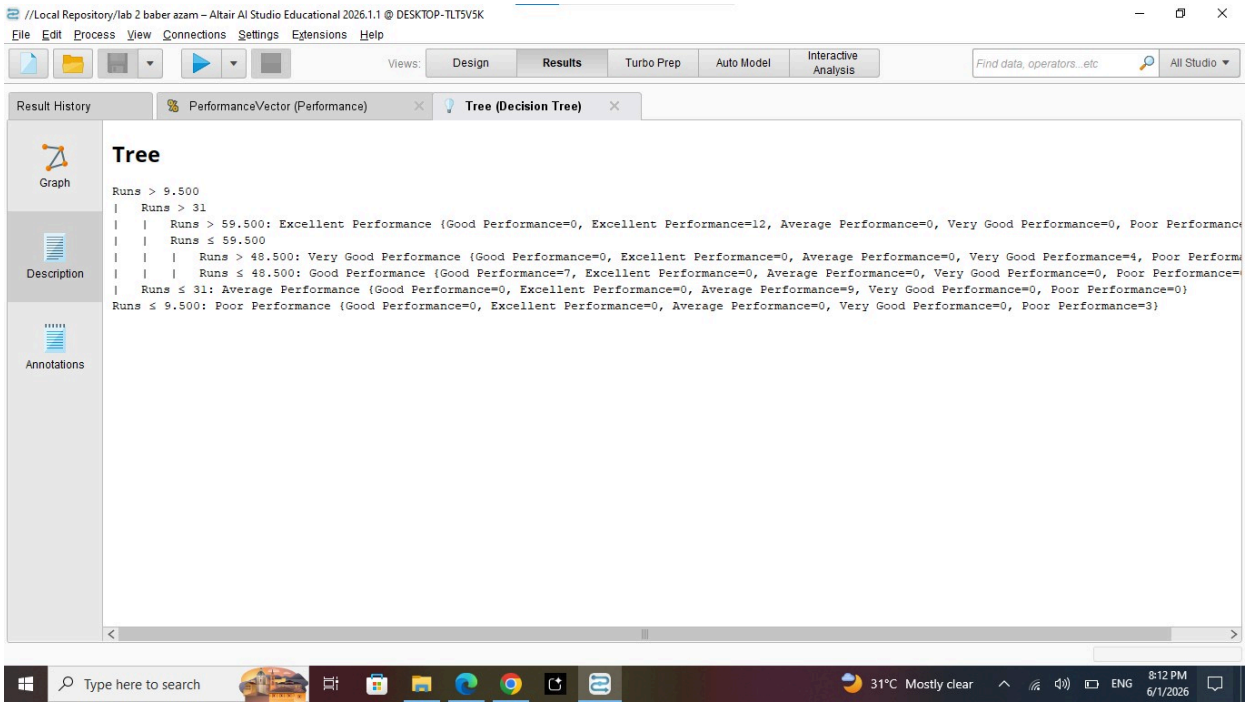
This means the model first checks the economy rate of the player. If the economy rate is **low (better performance)**, then the player's performance is mostly classified as **Good**. If the economy rate is **high (poor performance)**, then the model predicts the performance as **Poor**.

After Economy Rate, the model uses the **Wickets** column as the next important factor. It further divides the data based on how many wickets are taken to improve the accuracy of classification.

Finally, the decision tree keeps splitting the data step by step until each group becomes clear and contains only one type of result. This helps the model make accurate and simple predictions for cricket performance classification.







Project has **five classes**:

- Excellent Performance
- Very Good Performance

- Good Performance
- Average Performance
- Poor Performance

the entropy formula should be written as:

$$Entropy(S) = -\sum_{i=1}^{5} p_i \log_2(p_i)$$

or explicitly,

$$Entropy(S) = -p(E)\log_2 p(E) - p(VG)\log_2 p(VG) - p(G)\log_2 p(G) - p(A)\log_2 p(A) - p(P)\log_2 p(P)$$

Where:

- $p(E)$ = Probability of Excellent Performance
- $p(VG)$ = Probability of Very Good Performance
- $p(G)$ = Probability of Good Performance
- $p(A)$ = Probability of Average Performance
- $p(P)$ = Probability of Poor Performance

Example

The dataset of 50 matches contains:

Suppose your dataset of 50 matches contains:

Class	Count
Excellent Performance	15
Very Good Performance	8
Good Performance	12
Average Performance	10
Poor Performance	5

Then:

$$p(E) = \frac{15}{50} = 0.30$$

$$p(VG) = \frac{8}{50} = 0.16$$

$$p(G) = \frac{12}{50} = 0.24$$

$$p(A) = \frac{10}{50} = 0.20$$

$$p(P) = \frac{5}{50} = 0.10$$

Therefore,

$$Entropy(S) = -(0.30 \log_2 0.30 + 0.16 \log_2 0.16 + 0.24 \log_2 0.24 + 0.20 \log_2 0.20 + 0.10 \log_2 0.10)$$

$$Entropy(S) \approx 2.23$$

This entropy value can be used by the **Decision Tree algorithm in Altair AI Studio** to calculate Information Gain and determine the best attribute for splitting the data.

i want to like this "Overall Datas

Overall Dataset Entropy

Total Records = 50

Excellent Performance = 15

Very Good Performance = 8

Good Performance = 12

Average Performance = 10

Poor Performance = 5

Entropy(S) = -(15/50 $\log_2$ 15/50 + 8/50 $\log_2$ 8/50 + 12/50 $\log_2$ 12/50 + 10/50 $\log_2$ 10/50 + 5/50 $\log_2$ 5/50)

Entropy(S) ≈ 2.23

Runs Category

High Runs (> 60)

Excellent = 15

Very Good = 0

Good = 0

Average = 0

Poor = 0

Total = 15

Pure node because all records belong to Excellent Performance.

Medium Runs (50–60)

Excellent = 0

Very Good = 8

Good = 0

Average = 0

Poor = 0

Total = 8

Pure node because all records belong to Very Good Performance.

Low Runs (30–49)

Excellent = 0

Very Good = 0

Good = 12

Average = 0

Poor = 0

Total = 12

Pure node because all records belong to Good Performance.

Very Low Runs (11–29)

Excellent = 0

Very Good = 0

Good = 0

Average = 10

Poor = 0

Total = 10

Pure node because all records belong to Average Performance.

Extremely Low Runs (≤ 10)

Excellent = 0

Very Good = 0

Good = 0

Average = 0

Poor = 5

Total = 5

Pure node because all records belong to Poor Performance.

Weighted Entropy and Information Gain

Since the performance class is directly derived from runs scored, the entropy after splitting on Runs is 0.

Information Gain (Runs) = 2.23

Therefore, Runs is selected as the root node of the Decision Tree.

Team

Different opponent teams contain mixed performance categories. Therefore, entropy values are relatively high and information gain is lower than Runs.

Venue

Home, Away, and Neutral venues contain mixed performance categories. Therefore, entropy values remain high and information gain is lower than Runs.

Model Evaluation

After training the Decision Tree model using the dataset, the model was evaluated on testing data. Since the performance categories are determined from runs scored, the model correctly classifies most records.

The Excellent and Poor performance classes achieve high prediction accuracy because they are clearly separated by run thresholds. Some confusion may occur between Good, Very Good, and Average classes when run values are close to category boundaries.

Overall, the model performs very well for predicting Babar Azam's ODI batting performance.

Rule Formation

Rule 1: IF Runs > 60 THEN Performance = Excellent

Rule 2: IF Runs >= 50 AND Runs <= 60 THEN Performance = Very Good

Rule 3: IF Runs >= 30 AND Runs <= 49 THEN Performance = Good

Rule 4: IF Runs >= 11 AND Runs <= 29 THEN Performance = Average

Rule 5: IF Runs <= 10 THEN Performance = Poor

Rules Formulation in Pure AI Logic

Propositional Logic

P1: Runs(High) → Performance(Excellent)

P2: Runs(Medium) → Performance(VeryGood)

P3: Runs(Low) → Performance(Good)

P4: Runs(VeryLow) → Performance(Average)

P5: Runs(ExtremelyLow) → Performance(Poor)

First Order Logic

∀x [Runs(x, high) → Performance(x, excellent)]

∀x [Runs(x, medium) → Performance(x, verygood)]

∀x [Runs(x, low) → Performance(x, good)]

∀x [Runs(x, verylow) → Performance(x, average)]

∀x [Runs(x, extremelylow) → Performance(x, poor)]

Sample Facts

match(1, england, lahore, 42).

match(2, srilanka, karachi, 78).

match(3, india, dubai, 15).

match(4, australia, rawalpindi, 105).

match(5, newzealand, auckland, 33).

match(6, westindies, multan, 61).

match(7, bangladesh, dhaka, 12).

match(8, southafrica, johannesburg, 89).

match(9, afghanistan, sharjah, 27).

match(10, zimbabwe, harare, 54).

Rules

```
performance(ID, excellent) :-
match(ID, _, _, Runs),
Runs > 60.

performance(ID, verygood) :-
match(ID, _, _, Runs),
Runs >= 50,
Runs =< 60.

performance(ID, good) :-
match(ID, _, _, Runs),
Runs >= 30,
Runs =< 49.

performance(ID, average) :-
match(ID, _, _, Runs),
Runs >= 11,
Runs =< 29.
```

performance(ID, poor) :-
match(ID, _, _, Runs),
Runs =< 10.

Queries

performance(1, X).

performance(ID, excellent).

performance(ID, verygood).

performance(ID, good).

performance(ID, average).

performance(ID, poor).